



力扣学习

Elegant $\text{\LaTeX}$ 经典之作

作者：Ethan Deng & Liam Huang

组织：Elegant $\text{\LaTeX}$ Program

时间：April 9, 2022

版本：4.3

自定义：信息



不要以为抹消过去，重新来过，即可发生什么改变。——比企谷八幡

目录

第 1 章 数据库基础语法	1
1.1 计算字符	2
1.2 函数	4
第 2 章 第一章基础知识	7

第 1 章 数据库基础语法

SQL 是 Structured Query Language 的缩写，中文翻译为“结构化查询语言”

定义 1.1 (SELECT 语句)

在 SQL 中，**SELECT** 语句是用于从数据库表中检索数据的核心指令。其基本语法结构由以下两个核心部分组成：

- 选什么 (**What to select**)：紧跟在 **SELECT** 关键字之后，用于指定需要查询的列名（字段）。
- 从哪里选 (**From where to select**)：紧跟在 **FROM** 关键字之后，用于指定数据所在的表名。

东邪的 tip 1.1

如果要选择多个关键字每一个关键字都要用，隔开。如果需要选择表中的所有列，可以使用通配符 *****。

东邪的 tip 1.2

如果要选择名字不重复的就用 **distinct** **distinct** 是形容词所以放在要选什么的前面。**distinct** 对他后面的都有效而不是只是对他后面一个位置的有效。必须紧紧跟着 **select**

东邪的 tip 1.3

不同的系统选择前五行语法不一样，甲骨文的是 **where rownum** 小于等于 5。如果想从要 6 到 10 行就 **limit 5 offset 5** **offset** 是偏离的意思也就偏离 5 的定语是从 1 偏离 5 也就是从第 6 个开始，他前面的就是还需要多少行。1 到 10 有 10 个数，1 到 5 有五个数所以 5-1 需要再加 1

 **笔记** 1. 物理位置 vs. 逻辑索引在 SQL 的底层逻辑里，它不把第一行看作“1”，而是看作距离起点的偏移量 (**Offset**) 为 0 的位置。2. 为什么你的“偏离 5”等于“第 6 行”？这就是你刚才理解最神的地方：如果你想从第 6 行开始看，你的 **Offset** (偏离量) 必须是 5。因为：0(起点) + 5(偏离) = 5(索引)。而索引为 5 的那一行，在人类数数时正好是第 6 个。

东邪的 tip 1.4

三种注释—行内注释，整行注释 **/* */**

定义 1.2 (排序)

order by 必须是最后的字句，

核心逻辑拆解

数据隔离：**SELECT** 决定了最后要把哪些“列”展示在屏幕上给用户看 [cite: 5, 2026-03-07]。

排序依据：**ORDER BY** 决定了数据库在后台检索数据时，用哪根“轴”来进行对比排序 [cite: 5, 2026-03-07]。

合法性：只要这一列存在于你 **FROM** 后面的那张表（或关联表）里，数据库就能抓到它来进行排序，即便你并没有要求显示它

 **笔记** 提示：通过非选择列进行排序

通常，**ORDER BY** 子句中使用的列将是为显示而选择的列。但是，实际上并不一定要这样，用非检索的列排序数据是完全合法的。

多列排序示例

```
/* 下面的代码检索 3 个列，并按其中两个列对结果进行排序——首先按价格，然后按名称排序。 */  
SELECT prod_id, prod_price, prod_name
```

```
FROM Products
ORDER BY prod_price, prod_name;
```

Listing 1.1: 使用列位置排序

```
SELECT prod_id, prod_price, prod_name
FROM Products
ORDER BY 2, 3;
```

这个就是先按照 price 排序再按照 name 排序也就是先价格再首字母。也就是按照 name 的首字母排序。用这个方法局限的是只能用 select 里面的列作为排序的依据。desc 可以按照降序排列他的作用只在他前面的东西。

定义 1.3 (where 用来过滤数据)

where 后面跟着限定条件比如 price=3.5; 记住 order by 得写在他的后面

定义 1.4 (between)

between 和 where 要一起用 where between 爱你的、

定义 1.5 (or+and+In)

where+and 可以进行高级筛选可以一个 where——多个 and, or 是或。and 的优先级比 or 高如果需要先进行 or 判断必须加括号。有 and 有 or 用括号。In (,) 需要有括号每两个不同的用, 隔开。也是 where in。not 否定跟着后面的条件在条件复杂的时候尤为有效

东邪的 tip 1.5

通配符用来匹配值一部分的特殊字符, 比如说含有 bean bag, like 得配合着 where 用, 立刻后面跟着 'fish'F

 笔记 下划线 (_) vs 百分号 (%)

- % 是“贪婪”的: 它能匹配 0 个、1 个或无数个字符。比如 '% inch' 能匹配 8 inch, 也能匹配 12 inch。
- _ 是“死板”的: 一个下划线就代表一个占位符。

看图 3 中的代码: LIKE '__ inch teddy bear' (注意这里是两个下划线)。

 作业

1. 为什么 12 inch 和 18 inch 能匹配?
因为 12 是两个字符, 刚好填满了那两个下划线的坑。
2. 为什么 8 inch 匹配不到?
因为 8 只有一个字符。如果你用了两个下划线 __, SQL 就会在心里数数: “一、二... 诶? 8 后面没字符了, 但我这里要求必须有两个字符才能过关”, 于是就把它剔除了。

东邪的 tip 1.6 (and 后面还需要写一遍判断主体 prod_descname)

```
select prod_name, prod_desc from Products where prod_desc like "
```

1.1 计算字符

输入

```
SELECT prod_id,
       quantity,
       item_price,
       quantity*item_price AS expanded_price
```

```
FROM OrderItems
WHERE order_num = 20008;
```

输出

prod_id	quantity	item_price	expanded_price
RGAN01	5	4.9900	24.9500
BR03	5	11.9900	59.9500
BNBG01	10	3.4900	34.9000
BNBG02	10	3.4900	34.9000
BNBG03	10	3.4900	34.9000

东邪的 tip 1.7

如何进行加减乘除，把运算完的起一个名字即可然后就可以搜索了

section 计算字符 计算字符涉及到了合并列等等操作要紧跟着 select

输入：

```
SELECT vend_name + ' (' + vend_country + ')'
FROM Vendors
ORDER BY vend_name;
```

输出：

Bear Emporium	(USA)
Bears R Us	(USA)
Doll House Inc.	(USA)
Fun and Games	(England)
Furball Inc.	(USA)
Jouets et ours	(France)

东邪的 tip 1.8

+ 可以用来拼接' ' 是一个整体

输入

```
SELECT RTRIM(vend_name) + ' (' + RTRIM(vend_country) + ')'
FROM Vendors
ORDER BY vend_name;
```

输出

```
Bear Emporium (USA)
Bears R Us (USA)
Doll House Inc. (USA)
Fun and Games (England)
Furball Inc. (USA)
Jouets et ours (France)
```

东邪的 tip 1.9

rtrim 函数可以去掉所有右边空格，比如这里去掉的是 `vend_nameRT + rimLfT`

输入

```
SELECT RTRIM(vend_name) + '_( ' + RTRIM(vend_country) + ' )'
       AS vend_title
FROM Vendors
ORDER BY vend_name;
```

输出

vend_title

Bear Emporium (USA)
 Bears R Us (USA)
 Doll House Inc. (USA)
 Fun and Games (England)
 Furball Inc. (USA)
 Jouets et ours (France)

东邪的 tip 1.10

as 赋予了新的列一个名字也就是合成的这个列也有名字了可以 select 了

1.2 函数

东邪的 tip 1.11

upper 函数会让所有字母都大写括号里面就是大写的对象

表 1.1: 常用的文本处理函数

函数	说明
LEFT() (或使用子字符串函数)	返回字符串左边的字符
LENGTH() (或使用 DATALENGTH() 或 LEN())	返回字符串的长度
LOWER()	将字符串转换为小写
LTRIM()	去掉字符串左边的空格
RIGHT() (或使用子字符串函数)	返回字符串右边的字符
RTRIM()	去掉字符串右边的空格
SUBSTR() 或 SUBSTRING()	提取字符串的组成部分 (见表 8-1)
SOUNDEX()	返回字符串的 SOUNDEX 值
UPPER()	将字符串转换为大写

东邪的 tip 1.12

soundex() 方便寻找拼写错误的, 但是发生想的比如如果录入的时候 Michel 打错了大成 Michelle 了那么就可以在 where 后面加上判断条件 where soundex(vend_name)=soundex(Michel)

字母拼起来不能用——要用 concat

东邪的 tip 1.13

```
select cust_id, cust_name, upper(CONCAT (left (cust_contact, 2), left(cust_city, 3))) as
user_logn from Customers
```

```
SELECT AVG(prod_price) AS avg_price
FROM Products;
```

输出：

```
avg_price
-----
6.823333
```

东邪的 tip 1.14

记住每一次运算完都要给新结果一个名字。avg 只能运算单列会忽略 null 值，

- 使用 COUNT(*) 对表中行的数目进行计数，不管表列中包含的是空值 (NULL) 还是非空值。
- 使用 COUNT(column) 对特定列中具有值的行进行计数，忽略 NULL 值。

东邪的 tip 1.15

group by 是和 where 配合操作目的是对一类目标形成筛选和操作。group by 在 where 后面在 order by 前面

笔记 GROUP BY 的使用注意事项：

- GROUP BY 子句中列出的每一项都必须是检索列或有效的表达式，但不能是聚集函数。也就是说，GROUP BY 后面应当写“具体的分组依据”，而不能写 COUNT(*)、SUM(price) 这类分组之后才得到的结果。
- 如果在 SELECT 中使用了表达式，例如 YEAR(order_date)，那么在 GROUP BY 中也必须写相同的表达式，而不能只按原列分组。
- 在很多 SQL 实现中，GROUP BY 中不能直接使用 SELECT 子句里定义的别名，因此通常应写原表达式，而不要写别名。
- 大多数 SQL 实现不允许对长度可变的大字段进行分组，例如文本型字段或备注型字段。

笔记 WHERE 与 HAVING 的区别：

- WHERE 用于在分组之前对原始记录进行筛选，决定哪些行能够进入分组，因此 WHERE 是“管行”的。
- HAVING 用于在分组之后对分组结果进行筛选，决定哪些分组能够保留下来，因此 HAVING 是“管组”的。
- 一般来说，WHERE 中不能使用聚集函数，而 HAVING 中可以使用聚集函数。

例题 1.1 下面的 SQL 语句先筛选工资大于 5000 的员工，再按部门编号分组，最后只保留人数不少于 3 的部门：

```
SELECT dept_id, COUNT(*)
FROM employee
WHERE salary > 5000
GROUP BY dept_id
HAVING COUNT(*) >= 3;
```

笔记 上述语句的执行过程可以理解为：

- 先由 WHERE salary > 5000 筛选出工资大于 5000 的员工记录；
- 再按照 dept_id 对这些记录进行分组；
- 最后由 HAVING COUNT(*) >= 3 保留员工人数不少于 3 的那些部门分组。

东邪的 tip 1.16

聚集函数如果和 GROUP BY 一起出现，默认就是“对每个组聚集”。所以聚类函数 (aggregate function) count all 默认就是 count 刚分好的组。

 **笔记** 虽然 SELECT 子句写在最前面，但 SQL 查询在逻辑上并不是先执行 SELECT。一般可理解为按如下顺序执行：

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

因此，COUNT(*) 虽然写在 SELECT 中，但如果语句里有 GROUP BY，它实际统计的是每个分组中的行数。

表 1.2: ORDER BY 与 GROUP BY 的区别

ORDER BY	GROUP BY
对产生的输出排序	对行分组，但输出可能不是分组的顺序
任意列都可以使用（甚至非选择的列也可以使用） 不一定需要	只可能使用选择列或表达式列，而且必须使用每个选择列表表达式 如果与聚集函数一起使用列（或表达式），则必须使用

东邪的 tip 1.17

用 group by 不能指望他能排好

表 1.3: ORDER BY 与 GROUP BY 的区别

ORDER BY	GROUP BY
对产生的输出排序	对行分组，但输出可能不是分组的顺序
任意列都可以使用（甚至非选择的列也可以使用） 不一定需要	只可能使用选择列或表达式列，而且必须使用每个选择列表表达式 如果与聚集函数一起使用列（或表达式），则必须使用

第 2 章 第一章基础知识

LeetCode 1757. 可回收且低脂的产品：本题要求从 `Products` 表中筛选出同时满足低脂与可回收条件的产品。标准的 SQL 语句如下：

```
SELECT product_id FROM Products WHERE low_fats = 'Y' AND recyclable = 'Y';
```

从数学视角来看，这等价于在一个由行记录组成的全集 U 中，寻找满足谓词逻辑 $P(x) \wedge Q(x)$ 的元素集合，其中 $P(x)$ 定义为 $x.low_fats = 'Y'$ ， $Q(x)$ 定义为 $x.recyclable = 'Y'$ 。该过程在集合论中对应子集的交集运算 $A \cap B$ ，并最终通过 `SELECT` 操作将结果映射（投影）到 `product_id`。SQL 中的 `FROM` $\rightarrow$ `WHERE` $\rightarrow$ `SELECT` 与数学中先定义论域、再进行条件过滤、最后提取属性的步骤逻辑高度契合。此外，SQL 中的字符串字面量必须使用单引号包围，以区别于数据库的列名或表名标识符。这种将数据库操作转化为集合运算的思维方式，是理解关系型数据库底层原理的关键。